Thickness and Temperature Analysis

Data Analysis Assignment

This report addresses the four assignment sections using `analysis.py`. The main results use all 2,169 records in `data/Example Dataset.csv`. The odd/even subsets and four synthetic datasets are used for the additional checks reported below.

Analysis approach. The workflow follows the assignment: inspect and fill the data, set the time index, calculate grouped statistics, draw plots, and compare fitted models. Individual records are retained for calculations; hourly means are used only for the time-comparison plots.

A shared fitting function, such as `fit(values, normal)` or `fit(values, norm2)`, keeps the main workflow short. Model definitions are separate, so the same fitting and comparison steps can be reused.

AI assistance. AI coding assistance ("vibe coding") was used to implement the workflow; organize functions; revise plot layouts; and draft documentation.

The basic infrastructure of reading, plotting is implemented in the first draft, then refined through discussion and code review. The design was checked via small datasets.

The fit process is implemented purely by discussion based on the infrastructure and code review, including simplifying configuration and keeping the histogram and fitted curves on the same axes. Thus outputs were checked using the validation steps in Section 5.

1 Data Preparation and Summary

1.1 Load the dataset and display the first five rows

The CSV was read with `pandas.read_csv`, skipping blank lines. Table 1 shows the first five data records in their original file order, before imputation. Original column names and capitalization are retained, including `Thickness` and `Temperature`.

Table 1: First five data records. Empty measurement cells denote missing source values.

Timestamp	Unit ID	Product ID	Thickness	Temperature
2024/11/21 20:42	Unit-1	type0		
2024/11/21 20:42	Unit0	type1	578	3.140849
2024/11/21 20:43	Unit1	type2	578	3.140849
2024/11/21 20:44	Unit2	type0	578	3.140849
2024/11/21 20:45	Unit3	type1	578	3.140849

1.2 Missing-data check and mean imputation

There are eight missing values in each measurement column, occurring in the same eight records. No missing values were found in the other columns. For each measurement, the replacement value is the mean of its 2,161 observed values, calculated over the complete column rather than by product:

$$\bar{x}_{\text{obs}} = \frac{1}{n_{\text{obs}}} \sum_{i \in I_{\text{obs}}} x_i, \qquad n_{\text{obs}} = 2161.$$

Table 2: Missing values and the column means used for imputation. Blank replacement entries mean no imputation was performed.

Column	Missing count	Replacement mean
Timestamp	0	
Unit ID	0	
Product ID	0	
Thickness	8	573.714484
Temperature	8	4.071783

Only missing thickness and temperature values were filled. Existing values, including zeros, were retained, and the source file was not overwritten. Mean imputation follows the assignment but reduces sample variation and can influence distribution fitting; imputed values are not additional measurements.

1.3 Convert the timestamp and set the index

The timestamp was parsed with the format `%Y/%m/%d %H:%M` and set as a `DatetimeIndex`. The preparation function preserves row order; the line-plot function stably sorts records within each product when drawing them. The records span 19 November 2024 at 21:38 to 10 January 2025 at 22:30, covering eight observed calendar dates. There are 1,030 distinct timestamps and 1,139 duplicate occurrences beyond the first. All records sharing a timestamp were retained; a timestamp was not treated as a unique record identifier.

1.4 Mean thickness and temperature by product

Grouping the cleaned dataset by `product_id` gives the results in Table 3. Every product has 723 records, including imputed values.

Table 3: Product-level means after global-column mean imputation.

Product ID	Records	Mean thickness	Mean temperature
type0	723	573.562736	4.079222
type1	723	573.804189	4.067337
type2	723	573.776527	4.068791

The product means are numerically close. These descriptive averages alone do not establish that the product distributions are statistically equivalent.

1.5 Daily maximum and minimum thickness by product

Records were grouped jointly by product and calendar day, using `pd.Grouper(freq="D")`. Table 4 includes all 24 observed product/day combinations. The minimum and maximum were calculated from individual cleaned records, without averaging repeated timestamps.

Table 4: Daily thickness extrema. Only dates with observations are shown.

	type0		type1		type2	
Date	Min	Max	Min	Max	Min	Max
2024-11-19	570	585	572	586	573	588
2024-11-20	564	590	561	590	562	600
2024-11-21	557	587	558	588	552	581
2024-11-22	555	584	551	590	551	587
2024-11-23	548	578	553	582	551	581
2024-12-02	565	602	563	597	565	599
2024-12-03	562	597	561	599	567	599
2025-01-10	556	584	558	584	558	584

Data interpretation. The dataset covers separated measurement periods rather than an uninterrupted time series. Dates without records have not been assigned zero thickness or interpolated daily statistics. The smallest observed thickness is 548 and the largest is 602. All calculations retain these values; being outside a boxplot whisker or a specification limit is not, by itself, a reason to delete a record.

2 Visualization

2.1 Timestamp views

The data cover eight separated dates over almost two months. A densely connected plot mixes irregular sampling, repeated times, product differences, and long gaps. Three views make these features easier to read without removing observations.

Full time overview. Figure 1 retains all cleaned records and the true calendar spacing, showing both measurement periods and long gaps. Each product has one color. Lines break at a change of date, repeated or non-increasing timestamps, or a gap longer than one hour. Duplicate observations remain as plotted points.

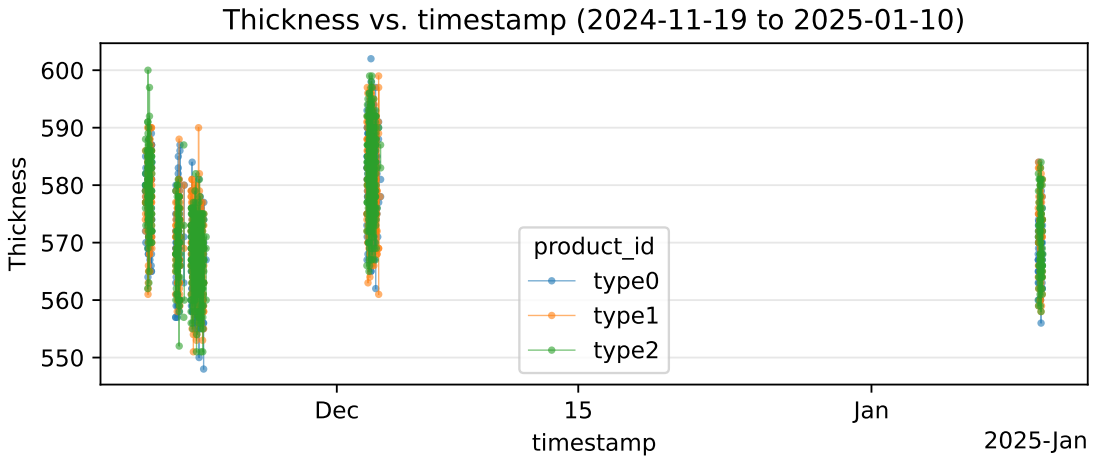


Figure 1: All products on a common real-time axis. Points represent individual measurements; blank periods reflect the source sampling.

Same hour across different dates. Mean thickness is calculated separately for each product, date, and hour. Fixing the hour in each panel compares similar times of day across dates and reduces dense overlapping points. Dates are equally spaced in this view and carry their actual date labels; the overview alone retains elapsed-time spacing. Figure 2 shows four representative hours. The complete generated chart contains all 23 observed hours.

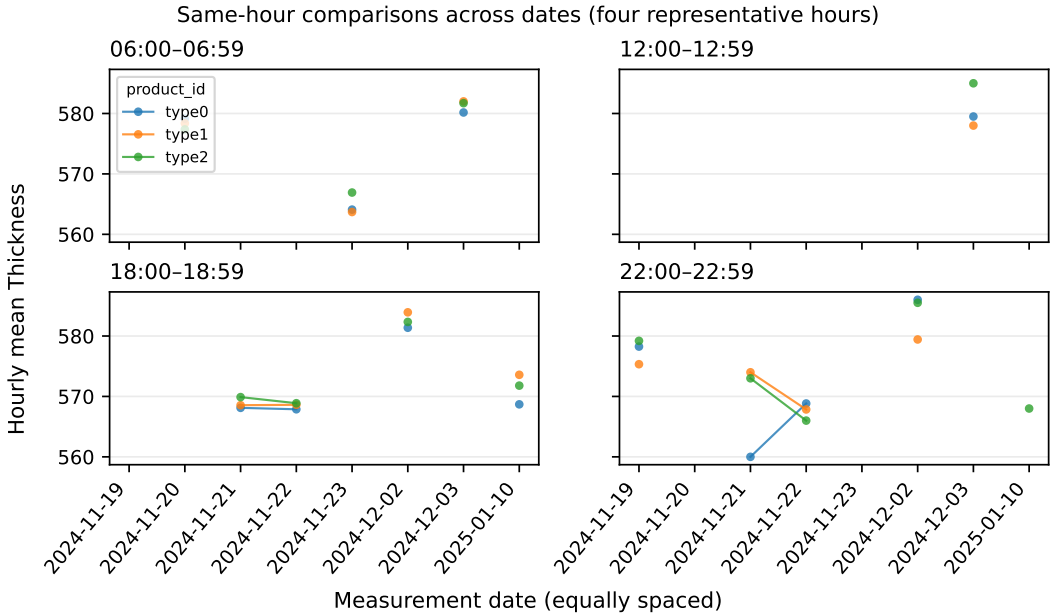


Figure 2: Hourly means at 06:00, 12:00, 18:00, and 22:00 across dates. Colors and thickness scale are shared; unmeasured combinations remain empty.

Different hours within the same date Figure 3 fixes the date in each panel and uses a common 0–23 hour axis. It uses the same 184 observed product/date/hour means as the cross-date view. This reveals within-day changes and different daily sampling windows. Missing hours remain gaps, without zeros or interpolation. Hourly averaging is only a display choice: individual cleaned records still supply all statistics, daily extrema, fits, and capability calculations.

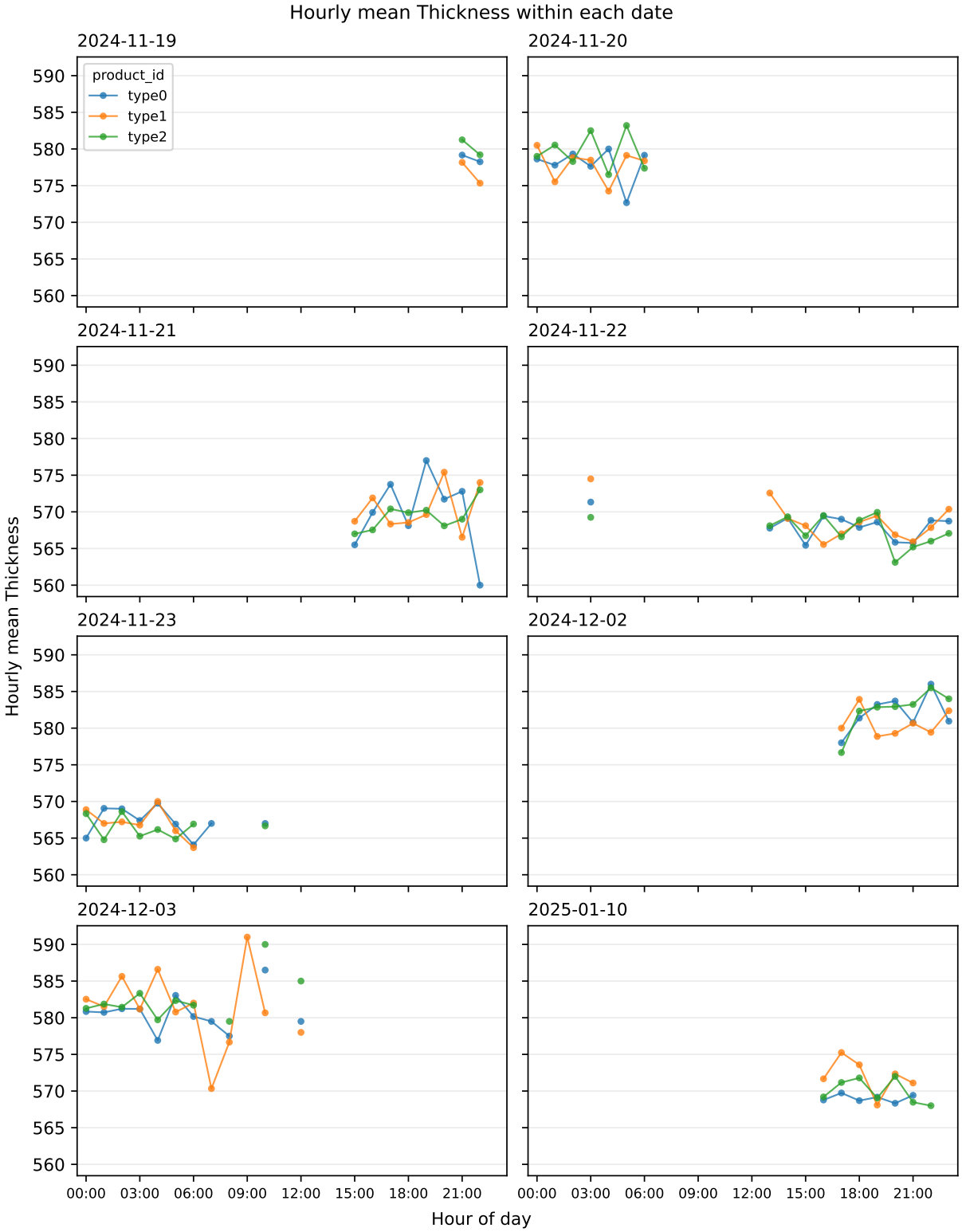


Figure 3: Hourly mean thickness within each measured date, with consistent product colors and a common vertical scale. Isolated points have no adjacent measured hours.

2.2 Overall thickness box plot

The box plot summarizes the thickness median, quartiles, and spread.

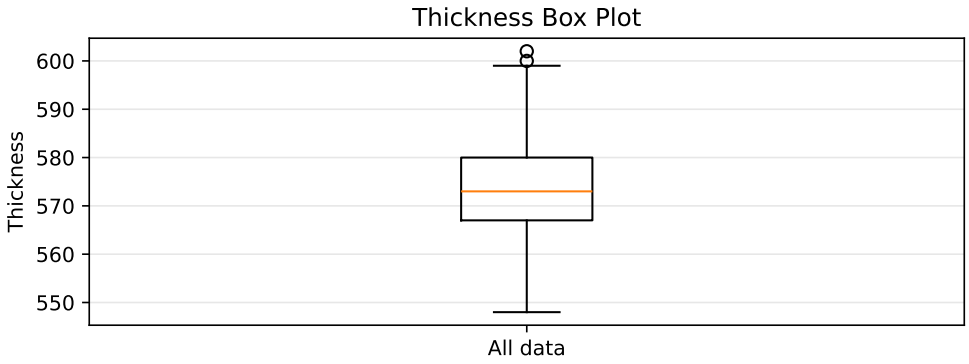


Figure 4: Thickness box plot for the complete mean-imputed dataset.

2.3 Temperature histogram

The histogram shows a right tail.

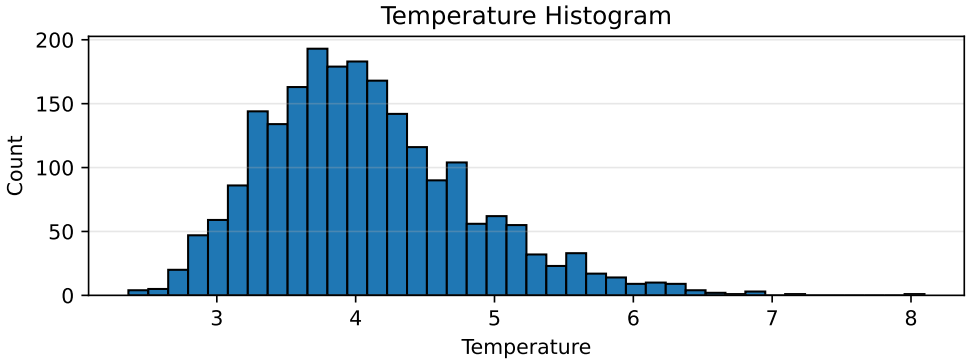


Figure 5: Temperature histogram.

2.4 Thickness box plots by product on the same figure

The three product distributions have similar central values and interquartile ranges. No formal equivalence test was performed.

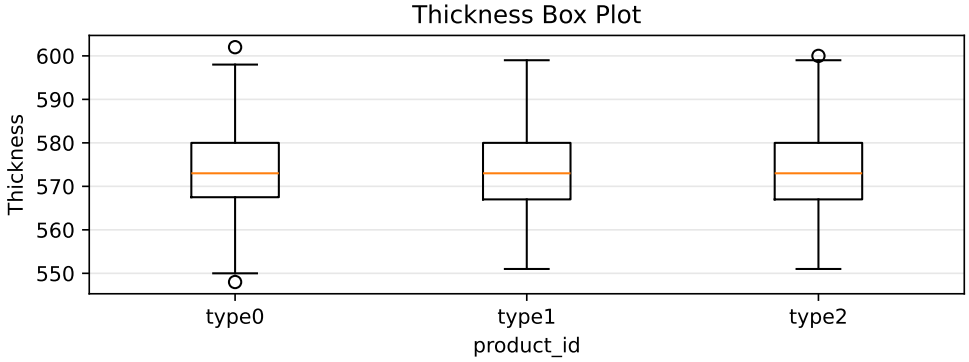


Figure 6: Thickness box plots for type0, type1, and type2 on the same scale.

3 Conventional Capability Calculations

3.1 Thickness capability for limits 550 and 650

Following the conventional formula in the assignment, define

$$C_{\text{pk}} = \min\left\{\frac{\bar{x} - \text{LSL}}{3s}, \frac{\text{USL} - \bar{x}}{3s}\right\}, \qquad s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}. \tag{1}$$

The standard deviation is the sample standard deviation, using `ddof=1` and all $n = 2169$ cleaned records. For thickness,

$$\bar{x} = 573.714484, \qquad s = 9.079214, \qquad \text{LSL} = 550, \quad \text{USL} = 650.$$

Consequently,

$$C_{\text{pk}} = 0.870651.$$

The lower specification limit controls the result. The distance from the mean to that limit is less than three sample standard deviations under this calculation.

3.2 Temperature limits for a target capability of 2

The mean and sample standard deviation of cleaned temperature are

$$\bar{T} = 4.071783, \qquad s_T = 0.743634.$$

Assume the two specification limits are symmetric about the current mean: $\text{LSL}_T = \bar{T} - d$ and $\text{USL}_T = \bar{T} + d$. Substitution into Equation (1) gives

$$C_{\text{pk}} = \frac{d}{3s_T} = 2 \quad \Longrightarrow \quad d = 6s_T.$$

Therefore, one solution is

$$\boxed{\text{LSL}_T = -0.390019, \qquad \text{USL}_T = 8.533585.} \tag{2}$$

4 Distribution Fitting and Cpk

4.1 Distribution comparison

Normal, Norm2, Norm3, and Lognormal were fitted to the same 2,169 thickness values after filling the missing data. Norm2 combines two normal distributions, and Norm3 combines three. The models were fitted using maximum likelihood and compared using BIC:

$$\mathrm{BIC} = k \log n - 2\ell.$$

Here, k is the number of parameters, n is the sample size, and ℓ is the fitted log-likelihood. BIC balances fit quality against extra parameters. A lower score is better.

Table 5: Model comparison, ordered by BIC.

Model	Parameters	Log-likelihood	BIC	ΔBIC
Norm2	5	-7827.827971	15694.066049	0.000000
Norm3	8	-7825.199918	15711.856008	17.789959
Lognormal	2	-7857.755063	15730.874169	36.808119
Normal	2	-7861.964635	15739.293312	45.227263

Norm2 has the lowest BIC and is selected. Its three extra parameters improve the fit enough to lower BIC by 45.23 compared with Normal. A third component improves log-likelihood by only 2.63 and increases BIC by 17.79, so the additional component is not retained.

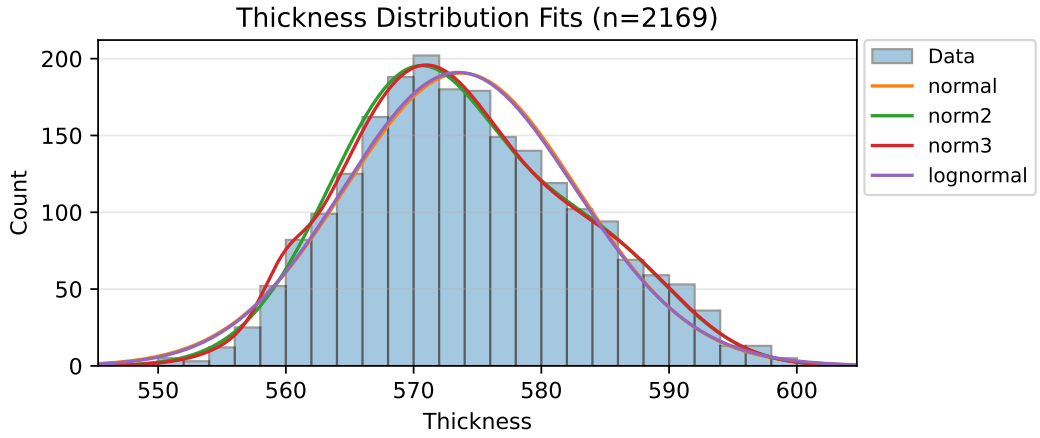


Figure 7: The four fits over the thickness histogram, using the same bins and axis limits.

4.2 Calculating Cpk from Norm2

Cpk was calculated using Norm2's fitted median m, lower percentile q_L (0.135%), and upper percentile q_U (99.865%):

$$C_{\mathrm{pk,pct}} = \min\left\{\frac{m - \mathrm{LSL}}{m - q_L},\ \frac{\mathrm{USL} - m}{q_U - m}\right\}. \tag{3}$$

Table 6: Norm2 results, using LSL = 550 and USL = 650.

Quantity	Value
Lower percentile q_L	550.283714
Median m	572.889548
Upper percentile q_U	599.735751
Cpk from the fitted distribution	**1.012550**

This calculation uses Norm2's fitted shape rather than the overall spread in Section 3. The lower percentile is close to LSL, so a good overall fit alone does not guarantee an accurate tail estimate.

5 Implementation and Result Validation

Synthetic samples and odd/even subsets were used to check model behavior and result consistency.

5.1 Synthetic distribution checks

Four samples with known generating distributions were used to check the same fitting function on different data shapes. Each sample contains 100 observations. The files were generated in sequence using `numpy.random.default_rng(42)` with these settings:

- Normal: $\mu = 600$, $\sigma = 10$.
- Norm2: $\mu = (580, 620)$, $\sigma = (5, 5)$, $w = (0.5, 0.5)$.
- Norm3: $\mu = (570, 600, 630)$, $\sigma = (4, 5, 4)$, $w = (0.3, 0.4, 0.3)$.
- Lognormal: $\log X \sim N(\log 600, 0.25^2)$.

Table 7: BIC for each fixed 100-observation sample. Bold values identify the selected model; compare candidates within each row.

Generating family	Normal	Norm2	Norm3	Lognormal
Normal	**701.964**	709.006	709.788	702.161
Norm2	899.078	**768.074**	772.481	898.897
Norm3	916.424	877.219	**813.294**	917.333
Lognormal	1308.787	1314.268	1326.081	**1300.810**

BIC selects the generating family in all four examples. For the normal sample, Normal and Lognormal differ by only 0.20 BIC units, so the two fits are difficult to distinguish. The non-selected Norm3 fit also reaches a standard-deviation lower bound, flagging a narrow extra component. These fixed examples check the implementation; repeated simulations would be needed to measure how often each family is selected.

5.2 Odd/even data comparison

Odd-numbered source records form the test dataset; even-numbered records form the validate dataset, excluding the header and blank lines. The files match `iloc[0::2]` and `iloc[1::2]` of the complete dataset exactly. Both subsets cover the measurement periods and have nearly balanced product counts.

Missing values were filled separately within each subset, then all four models were refitted. This checks whether the analysis gives similar results when different observations are used.

Table 8: Full-data and independently analyzed odd/even results. Standard deviations use `ddof=1`. Thickness limits are 550 and 650.

Quantity	Full	Odd: test	Even: validate
Records	2169	1085	1084
Missing thickness / temperature	8 / 8	4 / 4	4 / 4
Thickness mean	573.714484	573.914894	573.513889
Thickness standard deviation	9.079214	9.073408	9.084796
Conventional $\widehat{C_{\mathrm{pk}}}$	0.870651	0.878571	0.862756
Temperature mean	4.071783	4.073004	4.070561
Temperature standard deviation	0.743634	0.740302	0.747293
Lowest-BIC model	Norm2	Norm2	Norm2
Fitted lower percentile	550.283714	550.692078	549.846143
Fitted $\widehat{C_{\mathrm{pk,pct}}}$	1.012550	1.030905	0.993265

What the comparison shows. The thickness means differ by 0.401005 and the standard deviations by 0.011388. Both subsets select Norm2, showing consistent overall statistics and model choice.

The fitted Cpk values are 1.030905 and 0.993265, on opposite sides of 1. The test subset has minimum thickness 550, while validate includes 548. This shows that the tail-based Cpk is sensitive to the sample even when the mean and spread are similar.

The subsets share the same measurement periods. They check consistency within this dataset, rather than performance on a new batch or future period.

5.3 Reproducing the results

For the full-data analysis, set these constants in `analysis.py`:

```
CSV_FILE = BASE_DIR / "data/Example Dataset.csv"
OUTPUT_DIR = BASE_DIR / "data/output"
```

With the environment configured as in the README, run from the project directory:

```
.venv/bin/python analysis.py
```

Each dataset uses a separate output directory. The script prints numerical results and saves PNG figures. The report's JSON snapshots and PDF figures were prepared separately using the same analysis functions. Full hourly charts are saved under `reports/figures/hourly/`. Tables show rounded values; calculations use the original precision.

6 Proposed Follow-up: Monte Carlo Checks

Monte Carlo simulation could provide an additional check of the likelihood comparison. Repeated samples could be generated from a fitted model and refitted to examine changes in likelihood, model selection, and Cpk.

This study was not performed within the available schedule. Model selection therefore uses BIC, supported by the fixed synthetic samples and odd/even comparisons above. The proposed repeated simulations are separate from the four completed synthetic examples.